MIA

Holger Lyng

December 2025

1 Smoothing and Interpolation

Purpose: These methods are used to model data. One (intepolation) estimate values between observed data points. The other (smoothing) is to reduce noise while maintaining the underlying signal structure.

How is this achieved? It is modeled as a weighted sum of basis functions, B-spline; for **interpolation**, the number of basis functions M equals the number N observed data points($M = N$), thus the estimated curve pass through all points, perfectly fit the data. While for **smoothing** $M < N$ to create a simpler less noisy curve.

Difference: Unlike segmentation or registration, these are local preprocessingn steps focused on **signal reconstruction** rather than aligning image or labeling tissues.

To do smoothing and interpolation, there are some common equation and formulation that are important to define. Firstly they are both linear regression problems, therefore, both try to predict an outcome from input:

$$y(x, w) = w_0 + w_1 \cdot x_1 + \cdots + w_D \cdot x_D \tag{1}$$

Where x_d are the locations for given values t_n, w are tunable weights that is estimated given our data t/x and basis function ϕ, D are the dimension of the data. The basis function ϕ are non-linear transformations of the data locations x, a more general form of the linear regression model above is given by:

$$y(x, w) = \sum_{m=1}^{M} w_m \phi_m(x) \tag{2}$$

where $\mathbf{w} = (w_0, \cdots w_{M-1})^T$ are M tunable parameters.
To determine the parameters $\mathbf{w}$ we minimize the energy function less squares:

$$E(\mathbf{w}) = \sum_{n=1}^{N} (t_n - \sum_{m=0}^{M-1} w_m \phi_m(x))^2 \tag{3}$$

By take the gradient of the energy we can find a close form solution for the optimal weight:

$$\mathbf{w} = (\mathbf{\Phi^T \Phi})\mathbf{\Phi}^T t \tag{4}$$

And in the case where N=M (interpolation):

$$\mathbf{w} = \mathbf{\Phi}^{-1} t \tag{5}$$

Cause (For any square matrix in existance):

$$(\mathbf{\Phi^T \Phi})\mathbf{\Phi}^T = \mathbf{\Phi}^{-1}$$

We can add regulation to the weight equation calculation w by adding $+\lambda\Omega^T\Omega$ after $\Phi^T\Phi$. To then predict the $\hat{t}$ we do:

$$\hat{t} = \mathbf{\Phi w} \tag{6}$$

Some people also use:

$$\hat{t} = St$$

where S, called the smoothing matrix is defined as:

$$S = \mathbf{\Phi}(\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T \tag{7}$$

The above steps describe how to do smoothing and interpolation mathematically. We do not, however, have define the non-linear basis function which is at the core of these problems. The basis functions used in this course and in this assigments is β-splines, from order 0 to 2, discrete cosine transforms can also be used as Φ. The B-spline function $\beta^p(x)$ is defined recursively:

$$\beta^0(x) = \begin{cases} 1, & -\frac{1}{2} < x < \frac{1}{2} \\ \frac{1}{2}, & |x| = \frac{1}{2} \\ 0, & \text{otherwise,} \end{cases} \tag{8}$$

and for $p \geq 1$, the higher-order B-spline is defined by convolution:

$$\phi_m(x) = \beta^p(x) = \underbrace{(\beta^0 * \beta^0 * \cdots * \beta^0)}_{(p+1) \text{ times}}(x). \tag{9}$$

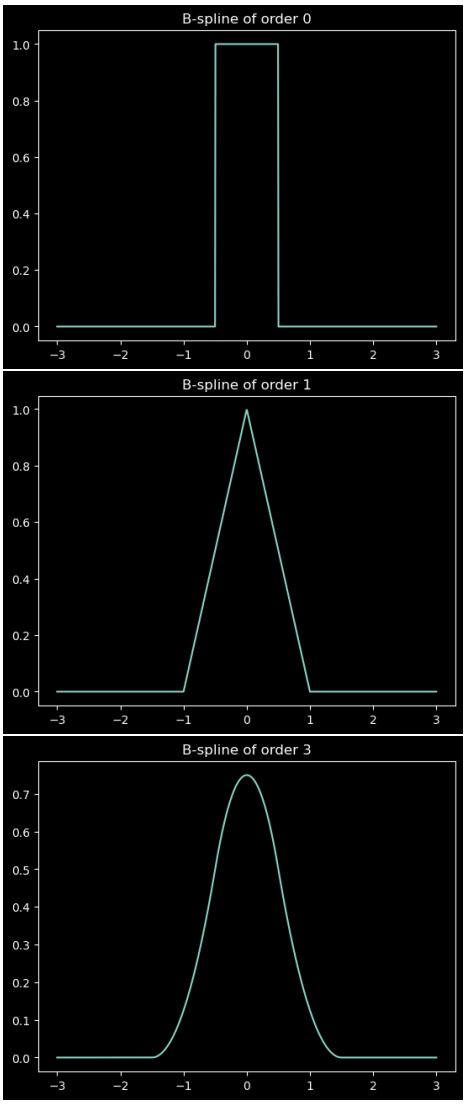


Figure 1: Basis function: B-spline

Now we continue to go into depth on smoothing.

1.1 Smoothing 1D

Smoothing is done when $M < N$, in other words, when we have less basis functions than we do datapoints. We primary use smoothing when we want to remove noise #denoise. Could probably also be used to compress an image.
A small note is that smoothing acts kind of like a lowpass filter, the less basis

we use, the more high frequency we "filter".

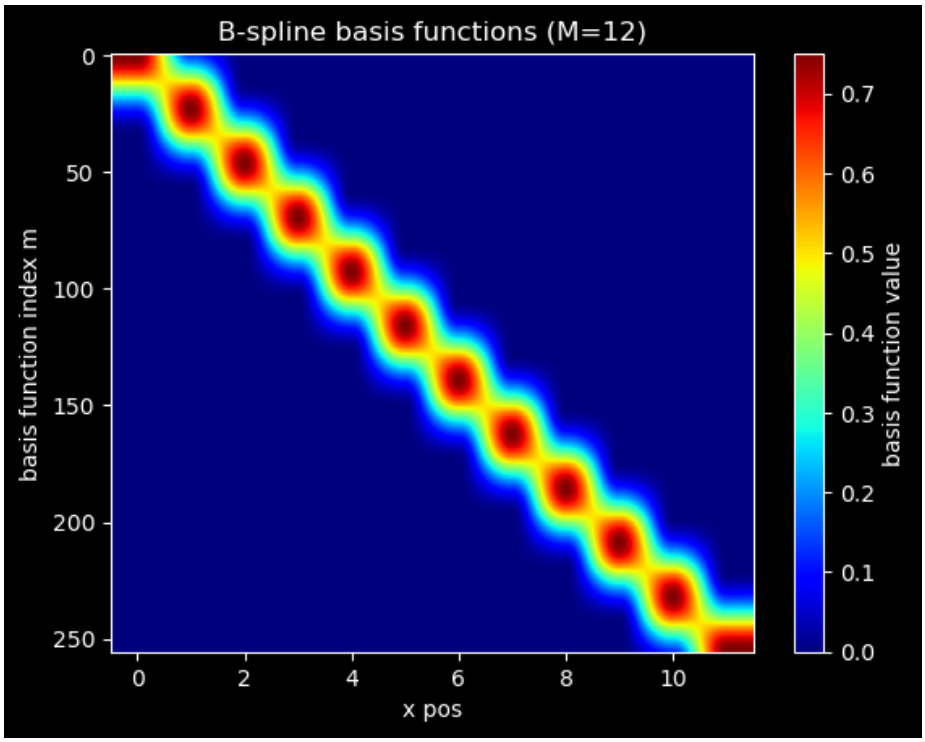


Figure 2: B-splines at location x

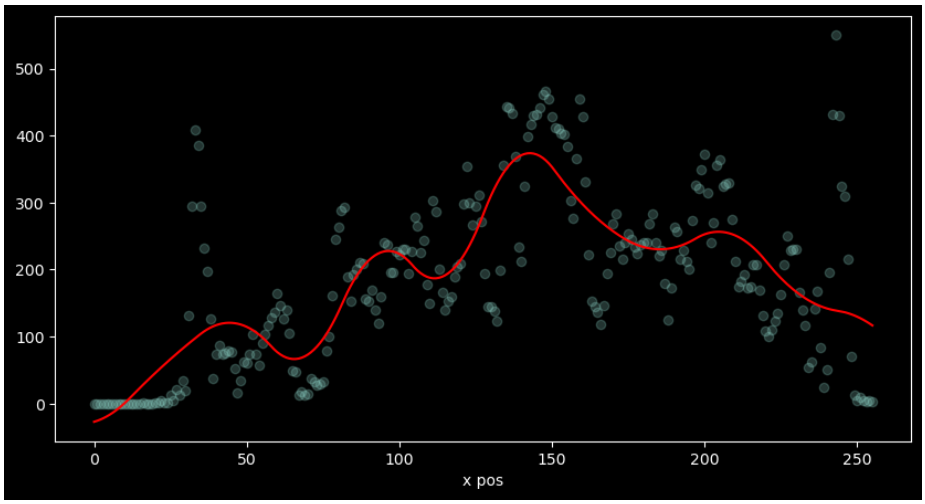


Figure 3: Smoothed 1D data

1.2 Smoothing 2D

If we ignore the way of doing it with a kroniker delta. Then smoothing can be written as:

$$\hat{T} = S_1 T S_2^T$$

Where each S, can be found by eq. 7:

$$S = \mathbf{\Phi}(\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T$$

Another way is to find the 2D weight matrix by:

$$W = (\mathbf{\Phi}_1^T\mathbf{\Phi}_1)^{-1}\mathbf{\Phi}^T\mathbf{T}\mathbf{\Phi}_2(\mathbf{\Phi}_2^T\mathbf{\Phi}_2)^{-1}$$

And then use:

$$\hat{T} = \mathbf{\Phi}_1\mathbf{W}\mathbf{\Phi}_2^T$$

For inference.

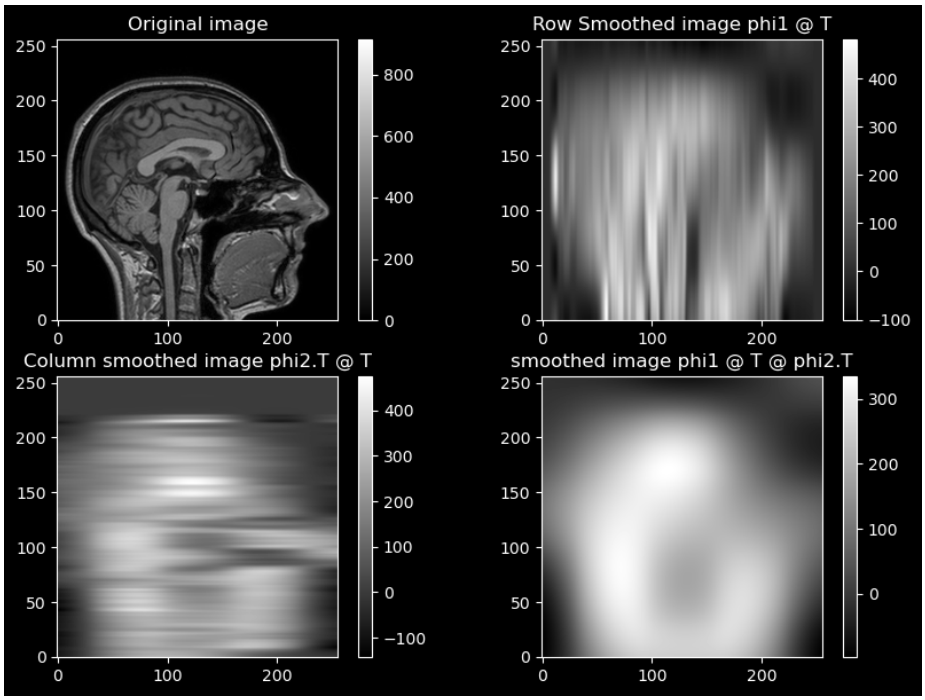


Figure 4: 2D image smoothed by exploiting separability

1.3 Interpolation 1D

Interpolation is done when $M = N$, in other words, when we have equal amounts of basis function(M) and data points(N). The purpose of interpolation is to "guess" what data points lie between the observed data points. When $M = N$ our regression curve will go through each data point, therefore, our curve will fit our data perfectly, any estimation in between data points is interpolations of the data. We calculate the weights we can use equation (5), and use equation (6) to predict.

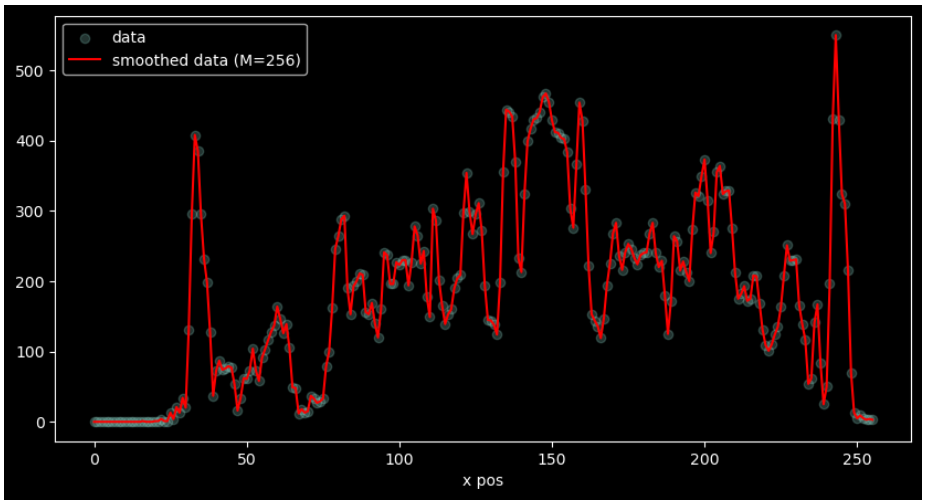


Figure 5: 1D interpolation $M = N$ and basis functions

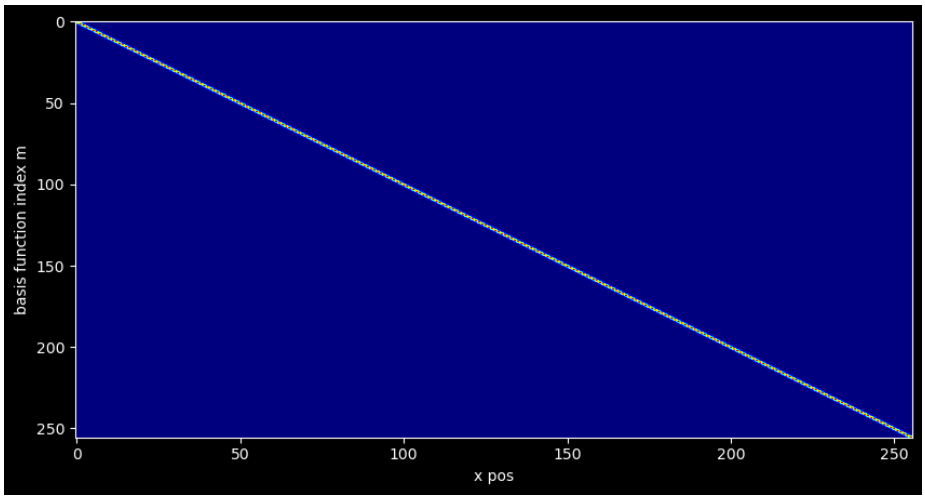


Figure 6: Basis-functions

1.4 Interpolation 2D

Interpolation in 2D, is a lttle different since we need two 1D basis function, one per dimension. The basis function are constructed in the same way as 1D, and the two 1D is combined:

$$\mathbf{\Phi} = \mathbf{\Phi_1} \otimes \mathbf{\Phi_2} \tag{10}$$

Since both vectors are of size M (which is equal to N, for interpolation), the combined size of $\mathbf{\Phi}$ is $M \times M$ matrix. The size of this matrix is problematic for

problem with 2 or 3 dimension, and M is large. Therefore, we exploit separability of basis functions $\mathbf{\Phi_1}$ and $\mathbf{\Phi_2}$. Instead of using equation 5, we can separate the basis function to get the same result faster and less computationally expensive:

$$\mathbf{w} = \mathbf{\Phi} t \tag{11}$$

instead we do:

$$\mathbf{w} = (\mathbf{\Phi_1^T \Phi_1})^{-1} \mathbf{\Phi^{-1} T \Phi_2} (\mathbf{\Phi_2^{-1} \Phi_2})^{-1} \tag{12}$$

Where if $\mathbf{\Phi_1}$ and $\mathbf{\Phi_2}$ are square matrices, equation becomes:

$$\mathbf{w} = \mathbf{\Phi_1^{-1} T} (\mathbf{\Phi_2}^{-1})^T \tag{13}$$

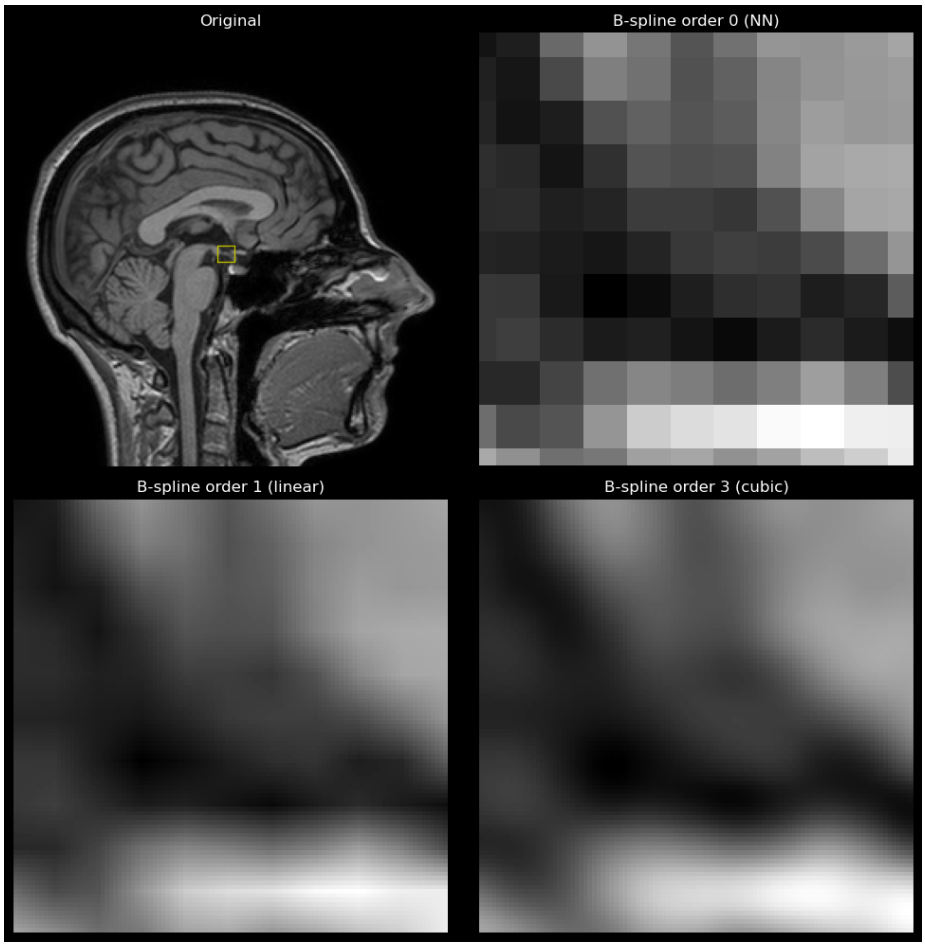


Figure 7: Caption

2 Landmark-based registration

Purpose: To spatilly align two images, fixed and moving, by using specific corresponding antomical points.

How is this achieved? Corresponding landmarks are manually identified in both images, and a transformation, either rigid or affine, is calculated to minimize the sum of squared distances between these points pairs.

Difference: Relies on human-selected points rather than image intensities or local pixel-wisedeformations. Global changes.

The purpose landmark-based registration is to align a moving image(MRI) to a fixed images(MRI). The moving and fixed image, are in voxel space, $\mathbf{v} = (v_1, v_2, v_3)$, two different voxel coordinates. When a MRI is acquired the scanner outputs an image $N_1 \times N_2 \times N_3$, an affine matrix($\mathbf{A}$) 3×3, and a translation vector 3×1 to map the voxel coordinates into world coordinate(mm):

$$\mathbf{x} = \mathbf{A}\mathbf{v} + \mathbf{t} \tag{14}$$

The general formulation for the formula above is given by:

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} & t_1 \\ a_{2,1} & a_{2,2} & a_{2,3} & t_2 \\ a_{3,1} & a_{3,2} & a_{3,3} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{M} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ 1 \end{pmatrix} \tag{15}$$

This M matrix is the affine matrix. The technique used here uses so-called homogeneous coordinates, vector are argumented with a 1 end. The homogeneous coordinates that absorbs the addition of the vector $\mathbf{t}$ into a single matrix multiplixation, which makes concatenating several affine matrices very easy. This leads us to the mapping of $\mathbf{v}_{T_1}$ to $\mathbf{v}_{T_2}$:

$$\begin{pmatrix} \mathbf{v}_{T2} \\ 1 \end{pmatrix} = \mathbf{M}_{T2}^{-1} \cdot \mathbf{M}_{T1} \cdot \begin{pmatrix} \mathbf{v}_{T1} \\ 1 \end{pmatrix}. \tag{16}$$

This formula above takes the affine matrix of T_1 and the inverse of T_2, then transforms the T_1 to world space, and then to T_2 voxel space, without any further alignment.

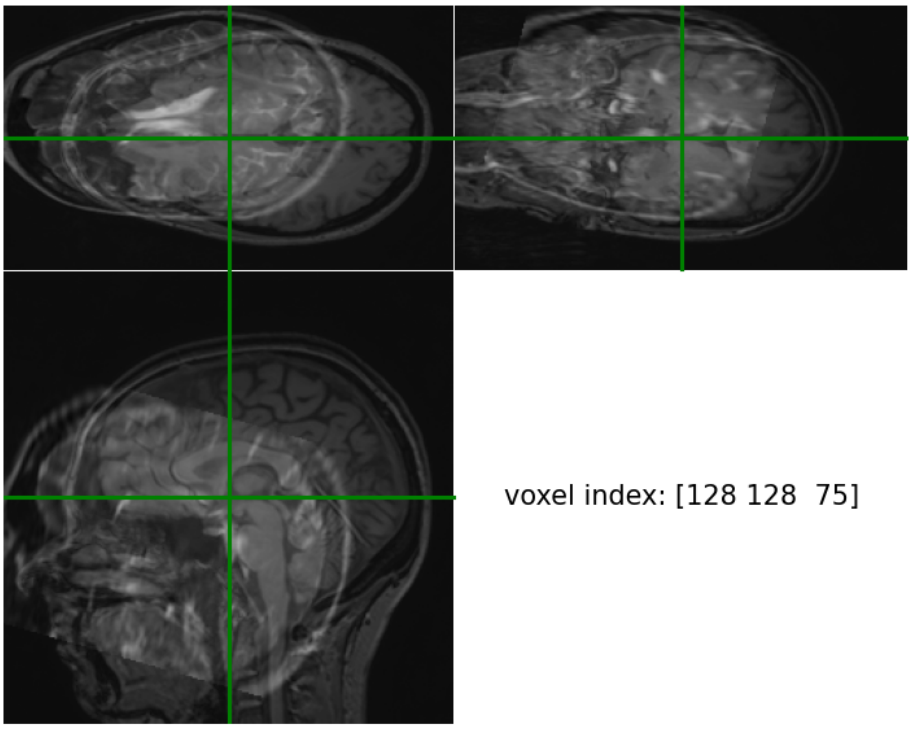


Figure 8: In same space but not aligned

When we have both images in the same voxel space, we can compute another affine matrix through the use of least squared, as explained below.

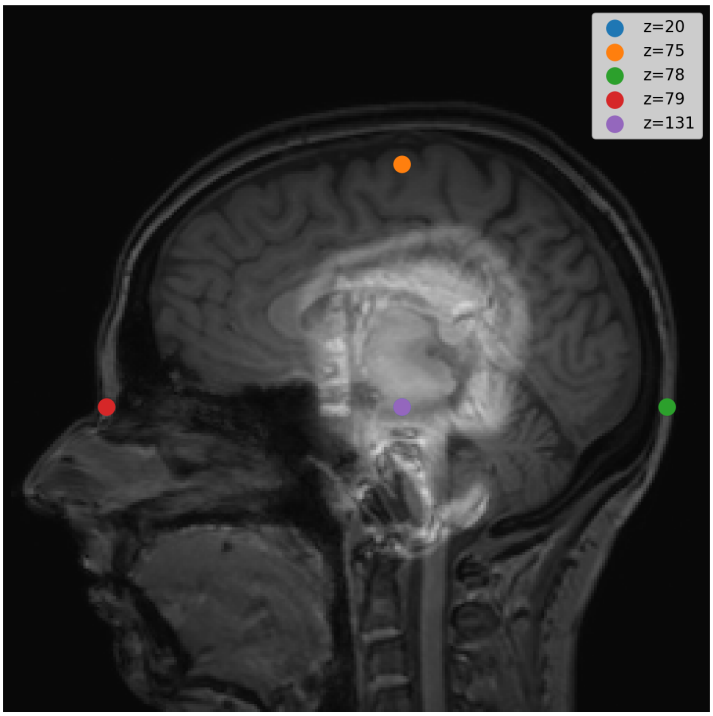
2.1 Fitting the imageass together using land-marks

Oh Hi (land) Mark
Once landmarks have been gathered (in the same space), we need to find some transform that can move the fixed image, to the moving image (confusing, i know). The way to do this in this course is least squares regression. We minimize the energy between our predicted landmark position from the fixed image after a transformation, and the actual position in the moving image:

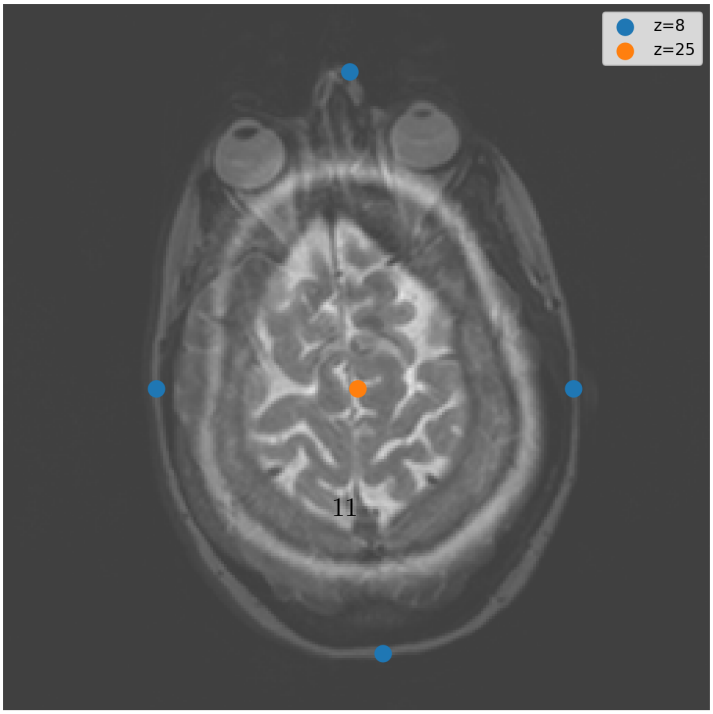
$$E(\mathbf{w}) = \sum_{i=1}^{I} ||\mathbf{y}_i - \mathbf{y}(\mathbf{x}_i, \mathbf{w})||^2$$

Where $\mathbf{y}_i$ is a landmark position in the moving image, and $\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ is a landmark predicted position after transformation to world space in the fixed image.

T1 Landmarks Combined



T2 Landmarks Combined



This is accomplised by, closed form solution for least squared:

$$\begin{pmatrix} t_d \\ a_{d,1} \\ \vdots \\ a_{d,D} \end{pmatrix} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T \begin{pmatrix} y_{1,d} \\ \vdots \\ y_{N,d} \end{pmatrix}, \tag{17}$$

where

$$\mathbf{X} = \begin{pmatrix} 1 & x_{1,1} & \cdots & x_{1,D} \\ 1 & x_{2,1} & \cdots & x_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N,1} & \cdots & x_{N,D} \end{pmatrix}.$$

The d-th row of the affine transformation parameters $\mathbf{A}$ and $\mathbf{t}$ is therefore given by equation (17). Doing this for all D dimensions yields all the required parameter values.
So, all row vectors in $\mathbf{X}$ are landmark coordinates for the fixed image and y is the moving. Doing this directly yields the vector seen in eq. 17, but using np.lstsq in python we get something different, which is almost already an affine transformation.
Once least squares has been done and all that, we can make the transformation matrix $\mathbf{M}$ by:

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} & t_1 \\ a_{2,1} & a_{2,2} & a_{2,3} & t_2 \\ a_{3,1} & a_{3,2} & a_{3,3} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\mathbf{M}} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ 1 \end{pmatrix}, \tag{18}$$

And the full transformation from fixed image voxel space to moving image voxel space is then:

$$\mathbf{v}_M = \mathbf{M}_M^{-1} \cdot \mathbf{M} \cdot \mathbf{M}_F \cdot \mathbf{v}_F, \tag{19}$$

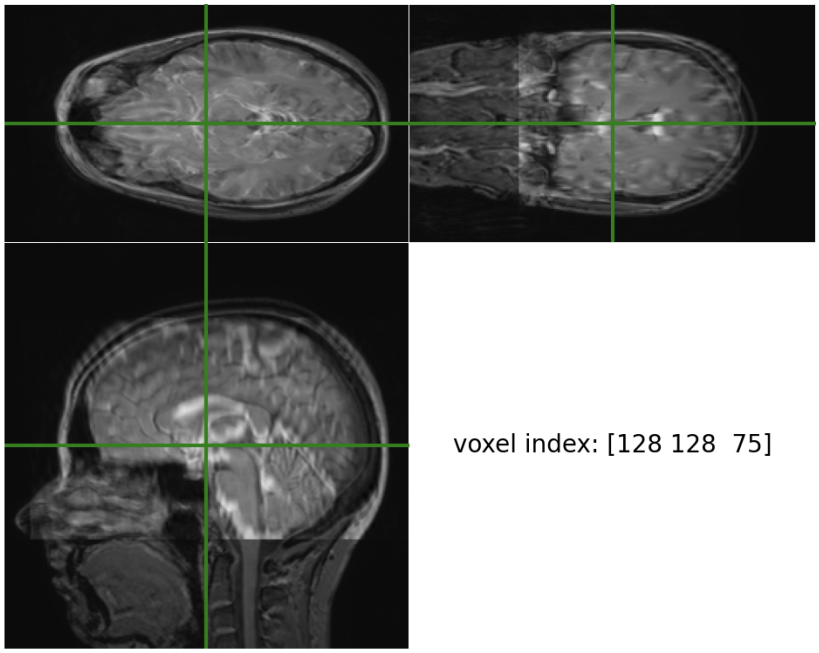


Figure 10: T1(fixed) and T2(moving) align with affine transformation

2.2 Rigid transformation

When finding the optimum $E(w)$ with respect to $\mathbf{w}$, we should be aware of some constraints that prevent us from decoupling the problem across dimension. There must be orthonormality of columns/rows:

$$\mathbf{R}^T\mathbf{R} = \mathbf{I} \tag{20}$$

and the determinant must be equal to one:

$$\mathbf{det}(\mathbf{R}) = 1 \tag{21}$$

These constraints preserve the length and the orthonormality of the matrix. This is not an issue for the translation vector t, therefore the same aproach as before is still application, the energy for a given $\mathbf{R}$:

$$E(\mathbf{w}) = \sum_{n=1}^{N} ||y_n - \mathbf{R}\mathbf{x}_n - t||^2 \tag{22}$$

When $E(\mathbf{w})$ is minimized when:

$$t = \bar{y} - \mathbf{R}\bar{\mathbf{x}} \tag{23}$$

Where $\bar{x}$ and $\bar{y}$ are the averages of the over the input landmark data fixed image, and the goal landmark location in the moving image, respectively. We can reformulate the energy:

$$E(\mathbf{w}) = \sum_{n=1}^{N} ||\tilde{y}_n - \mathbf{R}\tilde{x}_n||^2 \tag{24}$$

where $\tilde{x} = x_n - \bar{x}$ and $\tilde{y} = y_n - \bar{y}$, are now centered around the mean for fixed image landmarks and moving image landmarks respectively. We can use this to build a covariance matrix $\mathbf{H} = \tilde{\mathbf{x}}^T\tilde{\mathbf{y}}$. This covariance matrix $\mathbf{H}$, we can use do SVD (single value decomposition). From the SVD we get $\mathbf{U\Sigma V^T}$, $\mathbf{VU}^T = \mathbf{R}$ when $\mathbf{R}$ is calculated we can check the constraints, $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ and $\mathbf{det}(\mathbf{R}) = 1$, if the determinant is -1 then we can simply flip to 1. Once $\mathbf{R}$ is found and constraints is checked out, we compute the translation vecter t, with equation (23). Once that is done we can built the rigid transformation matrix M, like so:

$$\underbrace{\begin{pmatrix} r_{1,1} & r_{1,2} & r_{1,3} & t_1 \\ r_{2,1} & r_{2,2} & r_{2,3} & t_2 \\ r_{3,1} & r_{3,2} & r_{3,3} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\mathbf{M}}$$

From here we use equation (19) to transform from fixed image voxel space to moving image voxel space.

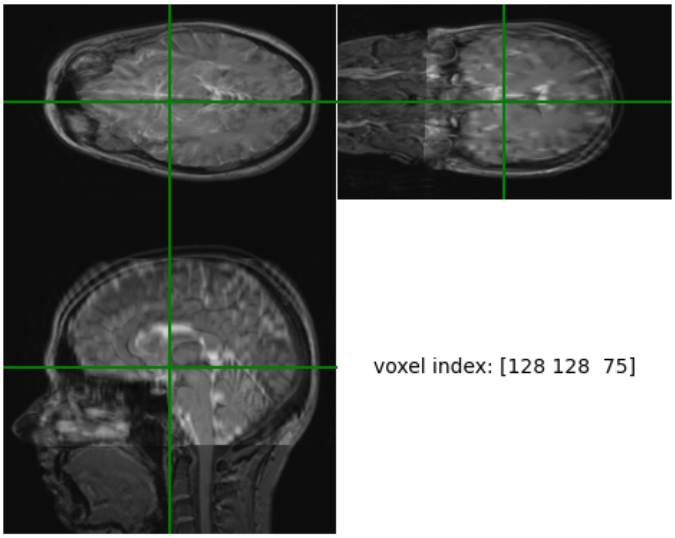


Figure 11: T1(fixed) and T2(moving) align with rigid transformation

2.2.1 Summery

What we are doing is first transforming both images from voxel coordinates to world coordinates, then minimizing the energy to find the affine transform matrix and translation vector from fixed to moving space, through equation (19), this is equation find the best possible alignment of the landmarks from fixed and moving image. With rigid we find the SVD values in world space, and compute the $\mathbf{R}$ and t, combine them to $\mathbf{M}$, then apply equation (19).

3 Intensity(Mutual information)

Purpose: To align images automatically by comparing pixel intensity patterns without needing manual landmarks and across modalities.

How is this achieved? An optimization algorithm iteratively adjusts transformation parameters to maximize a similarity measure, such as Mutual information, which is calculated using the joint histogram of the two images.

Difference: It is more objective and automated than landmark-based registration but is globally driven changes, unlike nonlinear which can deform images

locally.

p.s: In our implementation of the mutual information, there are some manual labour in term of determining which dimension we should rotate around to find optimal alignments. Also the actual transform is rigid, no shear ans scalling

Landmark-based registration suffers from several limitations, such as the need for manual annotation of landmarks and the fact that the registration is computed from only a small number of points. This limits both accuracy and robustness.

Intensity-based registration is another class of algorithms that can perform fully automated image registration. One such method is mutual information–based registration, which works well for multi-modal registration, for example registering CT to MRI or PET to MRI. Different imaging modalities typically have very different intensity characteristics.

In such situations, minimizing the sum of squared differences (SSD) is not appropriate, since the absolute intensity difference between a CT and an MRI image has little meaning due to their different intensity characteristics. Therefore, an alternative similarity measure, such as mutual information, is needed for fully automated multi-modal registration.

The basic idea behind mutual information, which originates from information theory, is to compute a joint histogram of the fixed and the moving image. In practice, this is done by iterating over all voxels in the fixed image and recording the corresponding intensity value at the same spatial location in the moving image. For each voxel pair, the bin corresponding to the intensity pair

$$(v_{\text{fix}}, v_{\text{moving}})$$

is incremented by one.

For example, if a voxel has intensity $v_{\text{fix}} = 0$ in the fixed image and $v_{\text{moving}} = 3$ in the moving image, the bin at position $(0, 3)$ in the joint histogram is incremented by one. Here, the first index corresponds to the intensity bin of the fixed image, and the second index corresponds to the intensity bin of the moving image.

3.1 Actual math

The joint is given by:

$$\mathbf{H} = \begin{pmatrix} h_{1,1} & \cdots & h_{1,B} \\ \vdots & \ddots & \vdots \\ h_{B,1} & \cdots & h_{B,B} \end{pmatrix}$$

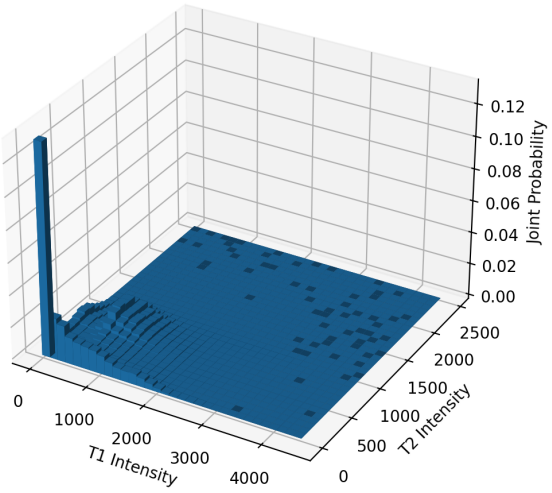


Figure 12: Joint histogram between the T1 and the resampled T2 image.

This joint histogram is filled out as explain a above. A key insight for registration is that the joint histogram with perfect alignment will have high bin count in in few bins, so most encountered intensity combinations (f, m) will be concentrated in a few bins. Whereas, if the images are badly align, this will have a smearing effect on the bins, e.g. more different intensity pair $\rightarrow$ more bins fill with low counts. We define this as the probability of seeing specific intensity combinations, like so:

$$E(\mathbf{w}) = H_{F,M}, \qquad \text{with} \qquad H_{F,M} = -\sum_{f=1}^{B}\sum_{m=1}^{B} p_{f,m}\log(p_{f,m})$$

Where $p_{f,m} = h_{f,m}/N$ normalized counts. $H_{f,m}$ is known as the joint entropy and measures how predictable a given intensity combination in fixed and moving image are. This probability will be lower, when the images are not well aligned and the histogram will display smearing.

One problem with this method is that the joint entropy will be zero if only the background of each image overlaps, therefore we need counter measure to ensure that the mutual information is information we are interested in good

looking at/aligning. Therefore, we introduce marginal entropies H_f and H_m:

$$H_f = -\sum_{f=1}^{B} p_f \log(p_f) \tag{25}$$

$$H_m = -\sum_{m=1}^{B} p_m \log(p_m) \tag{26}$$

Where $p_f = \sum_{m=1}^{B} p_{f,m}$, denotes the probability of encountering a voxel with intensity f. $p_m = \sum_{f=1}^{B} p_{f,m}$, denotes the probability of encountering a voxel with intensity m. We can add these to probabilities for the joint entropy term to get the mutual information term:

$$E(\mathbf{w}) = H_{f,m} - H_f - H_m \tag{27}$$

This ensure that we are looking at where the there is important information not just lowest joint entropy.

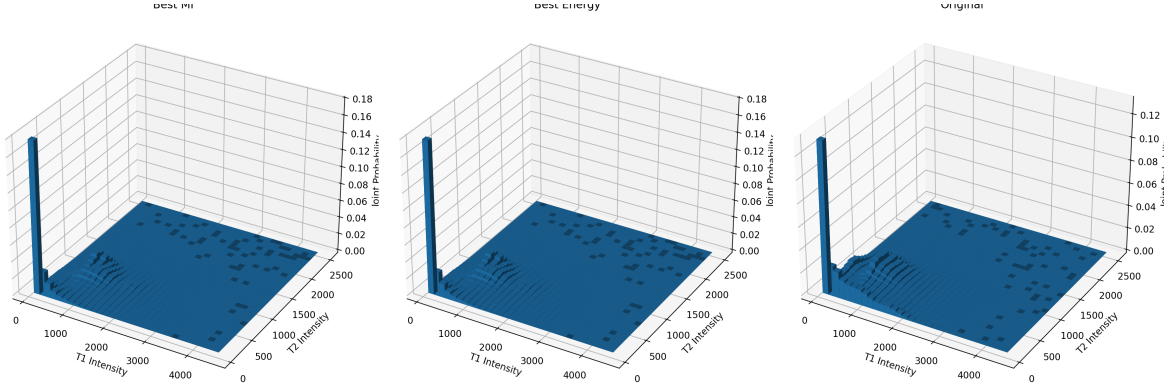


Figure 13: Overview of the joint histogram after registration.

4 Non-linear registration

Purpose: To account for local anatomical differences, e.g. brain swelling, organ movement, or size increase of tumors, that global rigid or affine transforms cannot fix.

How is this achieved? It defines a deformation function δ, which is a dense grid of basis functions with local weights to deform locally often regularized to ensure the deformation remains smooth and physically plausible.

Difference: While still intensity-based, it has far more degrees of freedom than the global rigid and affine models used in ex2 and ex3.

In certain situations, modeling tissue deformation is required. In such cases, the linear transformation model can be generalized to a non-linear deformation model. Typically, non-linear registration is applied only after global differences between images have been minimized. This is usually achieved by first transforming the moving image into the fixed image space, and then roughly aligning the images using a rigid or affine transformation.

The general non-linear deformation model is given by

$$y_d(\mathbf{x}, \mathbf{w}_d) = x_d + \delta_d(\mathbf{x}, \mathbf{w}_d), \tag{28}$$

where $d \in \{x, y, z\}$ denotes the spatial dimension and $\delta_d(\mathbf{x}, \mathbf{w}_d)$ represents the residual deformation in dimension d.

The residual deformation is modeled as a weighted sum of basis functions:

$$\delta_d(\mathbf{x}, \mathbf{w}_d) = \sum_{m=0}^{M-1} w_{d,m}\, \phi_m(\mathbf{x}), \tag{29}$$

where $\phi_m(\mathbf{x})$ are spatial basis functions and $w_{d,m}$ are the corresponding deformation parameters.

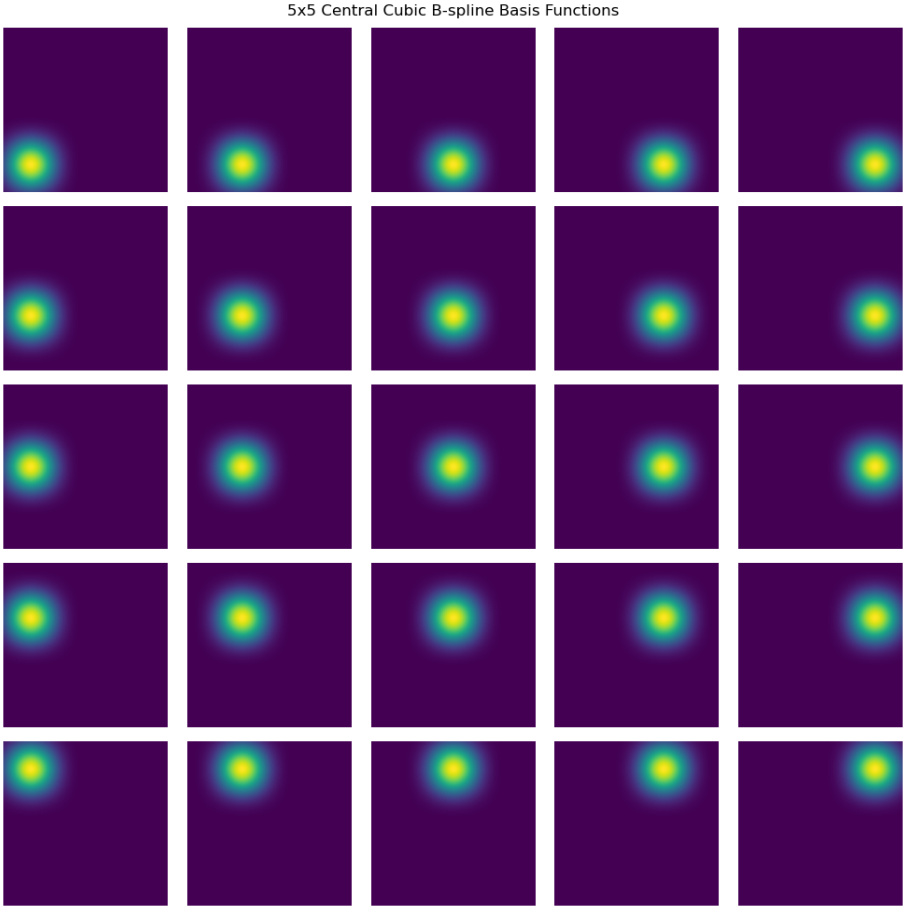


Figure 14: Central 5x5 basis subset.

This non-linear formulation allows the model to capture local, spatially varying deformations that cannot be represented by rigid or affine transformations alone.

The non-linear registration problem is typically solved using the Gauss–Newton optimization method, since the parameters $\mathbf{w}$ that minimize the energy function cannot be obtained in closed form. Standard least-squares optimization is not directly applicable because the image similarity term depends non-linearly on $\mathbf{w}$. Gauss–Newton addresses this by iteratively linearizing the image sampling function $\mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))$ with respect to $\mathbf{w}$ around the current parameter estimate.

The gauss-newtom method we will start with finding the partial derivatives, using the chain, by imploying a cubic β-spline interpolator model, these partial spatial derivatives can be conveniently computed everywhere. Because cubic

β-spline interpolates so that the images: is continuous everywhere, twich continuously differentiable and defined for all spatial position, not just grid points. The partial derivatives are given by:

$$\frac{\partial \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}{\partial w_{d,m}} = \frac{\partial \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}{\partial y_d} \frac{\partial y_d(\mathbf{x}_n, \mathbf{w}_d)}{\partial w_{d,m}} = g_{d,n}\, \phi_m(\mathbf{x}_n). \tag{30}$$

The partial derivative $\frac{\partial y_d(\mathbf{x}_n, \mathbf{w}_d)}{\partial w_{d,m}} = \phi_m(x)$, and $g_{d,m} = \frac{\partial \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}{\partial y_d}$ is introduced. Since $\mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ is non-linear, we linearize it by approximation with a first-order Taylor expansion:

$$\mathcal{M}(\mathbf{y}(\mathbf{x_n}, \mathbf{w} + \epsilon) \simeq \mathcal{M}(\mathbf{y}(\mathbf{x_n}, \mathbf{w})) + \sum_{d=1}^{D} \sum_{m=0}^{M} (g_{d,n} \phi_m(\mathbf{x}_n)) \epsilon_{d,m} \tag{31}$$

Here ϵ represents a small change to the parameter values $\mathbf{w}$, this asks the question how will the value of $\mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ change when the parameters is adjusted slightly?

But what should this small update be? To determine this we need to define two equation:

$$\tau_n = \mathcal{F}(\mathbf{x}_n) - \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w})) \tag{32}$$

Where τ per defination is the current residuals, $\mathcal{F}$ is the fixed image intensities and $\mathcal{M}$ are the sampled intensities from deformed moving image. Another usefull thing about τ is that $\mathbf{E}(\mathbf{w}) = \sum_{n=1}^{N} \tau^2$ is equal to sum of squared differences/Energy.

secondly, let us now take a look at the Taylor expansion eq:(31).

$$\mathcal{M}(\mathbf{y}(\mathbf{x_n}, \mathbf{w} + \epsilon) \simeq \mathcal{M}(\mathbf{y}(\mathbf{x_n}, \mathbf{w})) + \sum_{d=1}^{D} \sum_{m=0}^{M} (g_{d,n} \phi_m(\mathbf{x}_n)) \epsilon_{d,m} \tag{33}$$

This is a first-order approximation of the moving image intensity evaluated at the deformed location after a small update of the deformation parameters. And we want to find the residuals/sum of squared differences between this approximation and the constant fixed image intensities. So we add $\mathcal{F}(\mathbf{x}_n) -$.

$$\mathcal{F}(\mathbf{x}_n) - \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w} + \boldsymbol{\varepsilon})) \;\approx\; \mathcal{F}(\mathbf{x}_n) - \left[\mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w})) + \sum_{d=1}^{D} \sum_{m=0}^{M} (g_{d,n} \phi_m(\mathbf{x}_n)) \epsilon_{d,m} \right]. \tag{34}$$

One might notice that the middle part of this formulation looks awfully like the definition of τ.

$$= \underbrace{\mathcal{F}(\mathbf{x}_n) - \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}_{\tau_n} - \sum_{d,m} g_{d,n}\, \phi_m(\mathbf{x}_n)\, \varepsilon_{d,m}. \tag{35}$$

Therefore, we substitute in τ instead:

$$E(\mathbf{w}+\epsilon) \approx \mathcal{F}(\mathbf{x}_n) - \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}+\boldsymbol{\varepsilon})) \;\approx\; \sum_{n?1}^{N} \left[\tau_n + \sum_{d=1}^{D} \sum_{m=0}^{M} (g_{d,n}\phi_m(\mathbf{x}_n))\epsilon_{d,m} \right]. \tag{36}$$

We have now successfully made our non-linear problem into a linear regression problem with parameter ϵ. So we can find a closed-form solution for this problem. The value of ϵ minimizing $E(\mathbf{w}+\epsilon)$ is therefore given by:

$$\epsilon = (\mathbf{\Psi^T \Psi})^{-1} \mathbf{\Psi^T} \tau \tag{37}$$

1. Start with $\mathbf{w} = 0$, given weights and basis ϕ we compute the deformation δ. First iteration no changes are made since $x + 0 * \delta$, we just get x back
2. We also calculate the gradient using finite step differentials, we a small δ_{step} to sampled image: $g_x = \frac{(warped + \delta_{step_x}) - warped)}{\delta_{step_x}}$ and $g_y = \frac{(warped + \delta_{step_y}) - warped)}{\delta_{step_y}}$
3. Once the warped image is calculated we continue with calculating $\tau = original_{image} - warped_{image}$
4. We now use the gradient and ϕ to built $\Psi = (\mathbf{G_1 \Phi} | \cdots | \mathbf{G_D \Phi})$
5. Once we have calculated τ and Ψ we can calculate update parameter ϵ

We can a regularizer for ϵ, this is done to ensure that ϵ does not become to large, its called levenberg-Marquardt algorithm:

$$\epsilon = (\mathbf{\Psi^T \Psi} + \lambda \mathbf{I})^{-1} \mathbf{\Psi^T} \tau \tag{38}$$

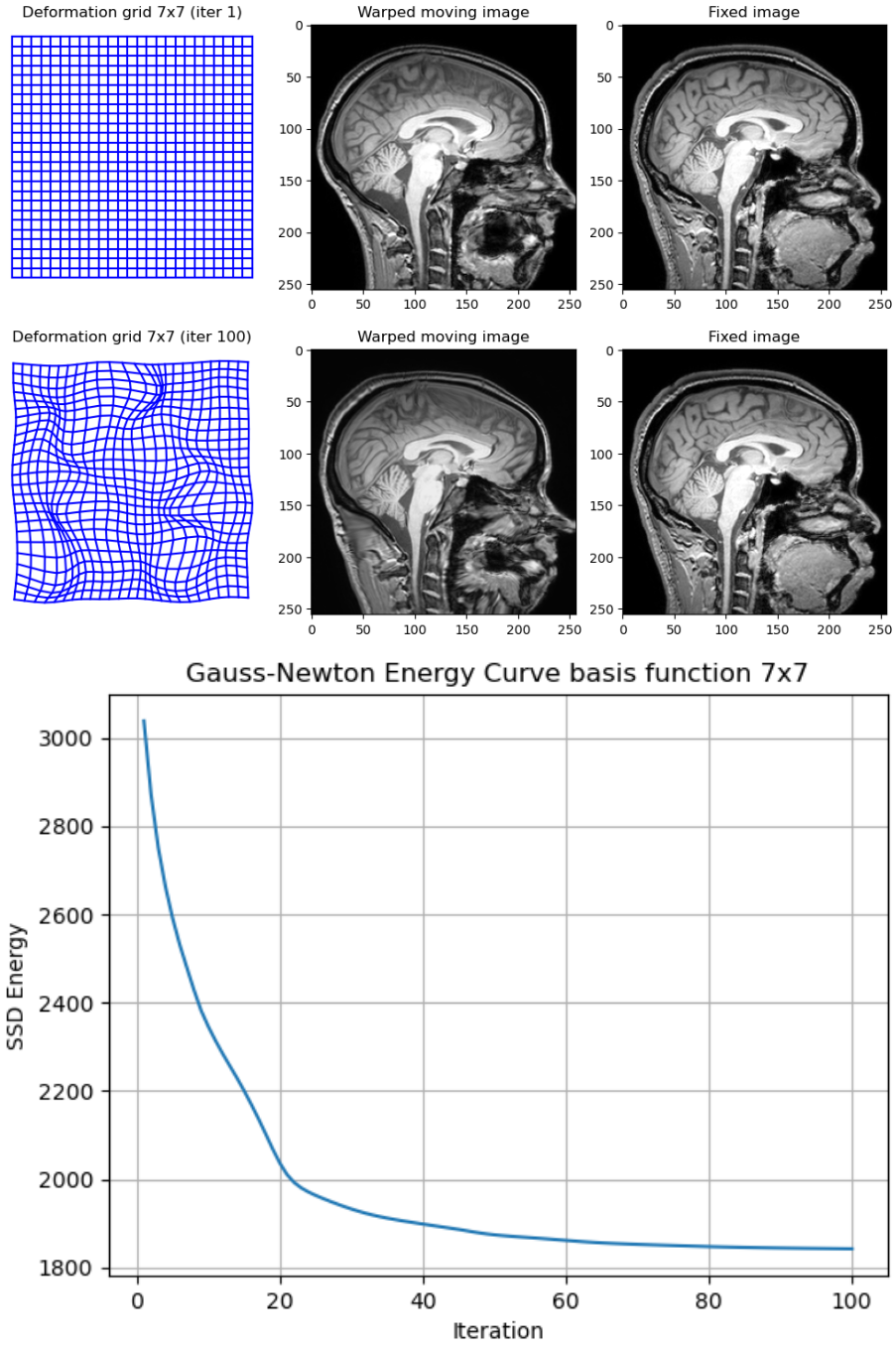


Figure 15: In this figure the results from simulation with 7x7 basis functions are shown. First the original deformed grid, moving image and fixed image are shown at the top. In the middle the deformed grid, warped moving image and fixed moving image, in the final iteration of Gaúss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 7x7 basis functions are shown.

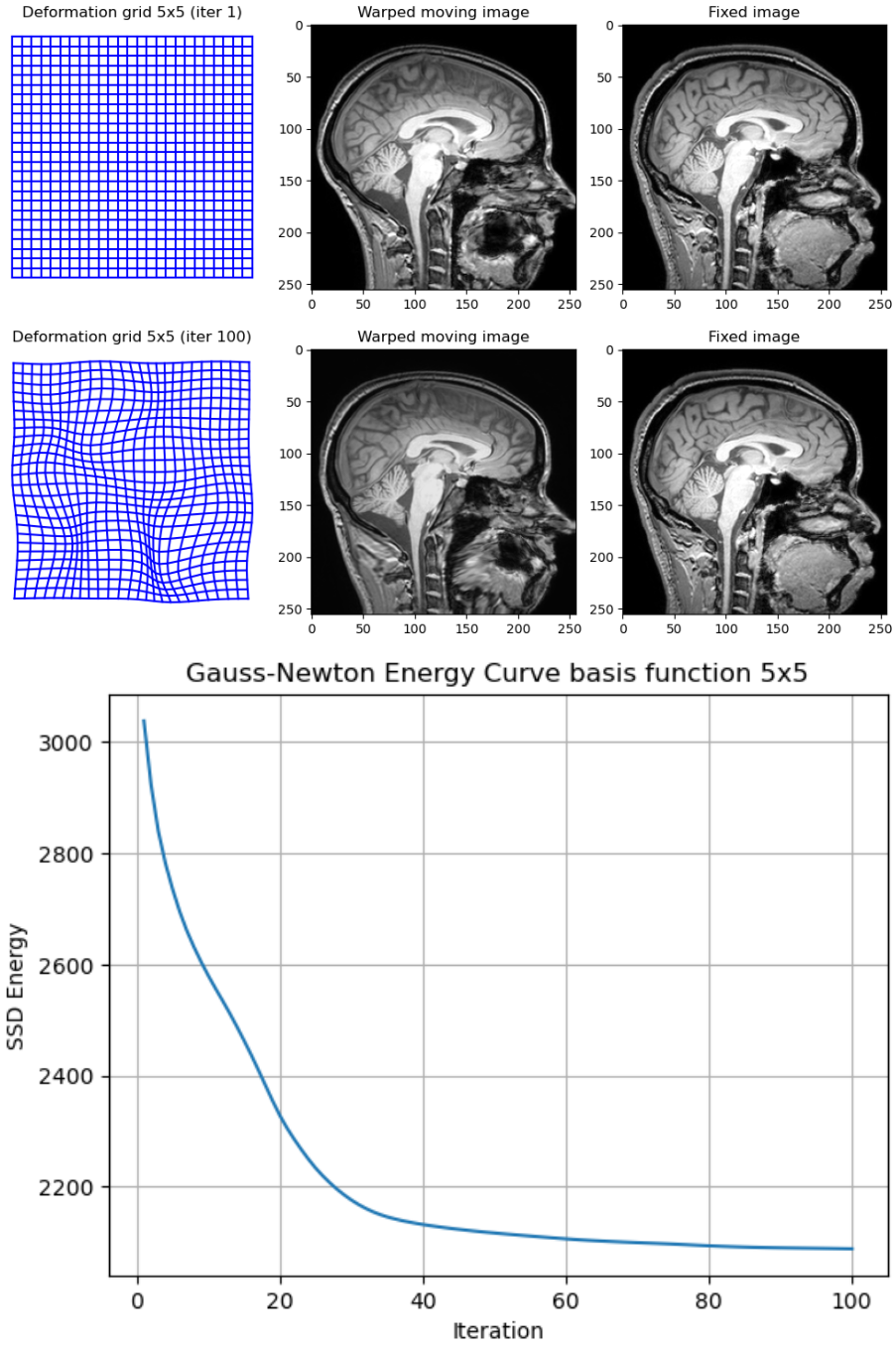


Figure 16: In this figure the results from simulation with 5x5 basis functions are shown. First the original deformed grid, moving image and fixed image are shown at the top. In the middle the deformed grid, warped moving image and fixed moving image, in the final iteration of Gaúss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 5x5 basis functions are shown.

5 Expectation–Maximization Algorithm

Purpose: To perform unsupervised tissue segmentation, delineate GM, WM and CSF, by fitting gaussian to the image intensities, e.g. fitting a statistical model to the image intensities.

How is this achieved? ex6 → done by fitting a feed-forward neural network with 9 input dimension, 6 basis functions(with 9 weights for each input), 6 hidden units(sigmoid activation) and one output.

Also: In ex7 → with a Unet structure (encoder-decoder) that down samples spatial dimension and increase channels(feature maps) to capture contextual information and skip-connection for fine-details.

Difference: These models are discriminative supervised methods; unlike EM algorithm, it requeires a groud-truth training set to learn, but can handle more complex patterns than simple intensity distributions.

In the previous sections, we considered the problem of registering two images to one another. In this section, we address a different but related problem: how to automatically delineate different tissue types in an image.

This problem can be formulated using a probabilistic framework based on Bayes' rule. Given an observed voxel intensity d and a tissue label l, the posterior probability of the label is given by

$$p(l \mid d, \hat{\boldsymbol{\theta}}) = \frac{p(d \mid l, \hat{\boldsymbol{\theta}}_d)\, p(l \mid \hat{\boldsymbol{\theta}}_l)}{p(d \mid \hat{\boldsymbol{\theta}})}. \tag{39}$$

The posterior probability $p(l \mid d, \boldsymbol{\theta})$ represents the probability that a voxel belongs to tissue class l given the observed intensity d and the current model parameters. It combines information from the data through the likelihood term and prior knowledge through the label prior, thereby providing a probabilistic tissue classification for each voxel.

The first term in the numerator, $p(d \mid l, \hat{\boldsymbol{\theta}}_d)$, is the likelihood and models the probability of observing intensity d given a tissue label l and its associated parameters. This term is typically modeled using a Gaussian or a Gaussian mixture model.

The second term, $p(l \mid \hat{\boldsymbol{\theta}}_l)$, is the prior distribution over tissue labels and encodes prior knowledge about the expected distribution of tissues, for example through spatial priors or smoothness constraints.

The denominator $p(d \mid \hat{\boldsymbol{\theta}})$ serves as a normalization constant and is independent of the label l.

Since the tissue labels are not observed, they are treated as latent variables. The parameters $\boldsymbol{\theta}$ are therefore commonly estimated using the Expectation–Maximization (EM) algorithm, which alternates between estimating the posterior distribution of the labels (E-step) and updating the model parameters (M-step).

5.1 Gaussian mixture

In this section, we define the components required to use Gaussian mixture models as inference tools for image segmentation. We begin by defining the segmentation prior.

$$p(\mathbf{l} \mid \boldsymbol{\theta}_l) = \prod_{n=1}^{N} p(l_n \mid \boldsymbol{\theta}_l), \tag{40}$$

with

$$p(l_n = k \mid \boldsymbol{\theta}_l) = \pi_k. \tag{41}$$

The parameter vector $\boldsymbol{\theta}_l = (\pi_1, \ldots, \pi_K)^T$ consists of the class probabilities π_k, where $\pi_k > 0$ and $\sum_{k=1}^{K} \pi_k = 1$. This model assumes that voxel labels are assigned independently and that each tissue class occurs with frequency π_k.

The likelihood model assumes that the intensity at each voxel depends only on the label at that voxel and not on neighboring voxels:

$$p(\mathbf{d} \mid \mathbf{l}, \boldsymbol{\theta}_d) = \prod_{n=1}^{N} p(d_n \mid l_n, \boldsymbol{\theta}_d). \tag{42}$$

The intensity distribution associated with each label k is modeled as a Gaussian distribution with mean μ_k and variance σ_k^2:

$$p(d_n \mid l_n = k, \boldsymbol{\theta}_d) = \mathcal{N}(d_n \mid \mu_k, \sigma_k^2). \tag{43}$$

The parameter vector $\boldsymbol{\theta}_d = (\mu_1, \ldots, \mu_K, \sigma_1^2, \ldots, \sigma_K^2)^T$ defines the Gaussian mixture components.

The denominator in Bayes' rule, $p(\mathbf{d} \mid \boldsymbol{\theta})$, is known as the evidence or marginal likelihood. It represents the probability that the model generates the observed image, independent of the unknown labels. It is obtained by marginalizing over all possible label configurations:

$$p(\mathbf{d} \mid \boldsymbol{\theta}) = \sum_{\mathbf{l}} p(\mathbf{d} \mid \mathbf{l}, \boldsymbol{\theta}_d) p(\mathbf{l} \mid \boldsymbol{\theta}_l). \tag{44}$$

Under the assumptions of conditional independence between voxels and independent label assignments, this expression factorizes as

$$p(\mathbf{d} \mid \boldsymbol{\theta}) = \prod_{n=1}^{N} \sum_{k=1}^{K} \pi_k \, \mathcal{N}(d_n \mid \mu_k, \sigma_k^2). \tag{45}$$

The per-voxel likelihood

$$p(d \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(d \mid \mu_k, \sigma_k^2) \tag{46}$$

is a weighted sum of Gaussian distributions, which explains why this model is referred to as a Gaussian mixture model. The mixing coefficients π_k represent the prior probability of selecting each Gaussian component.

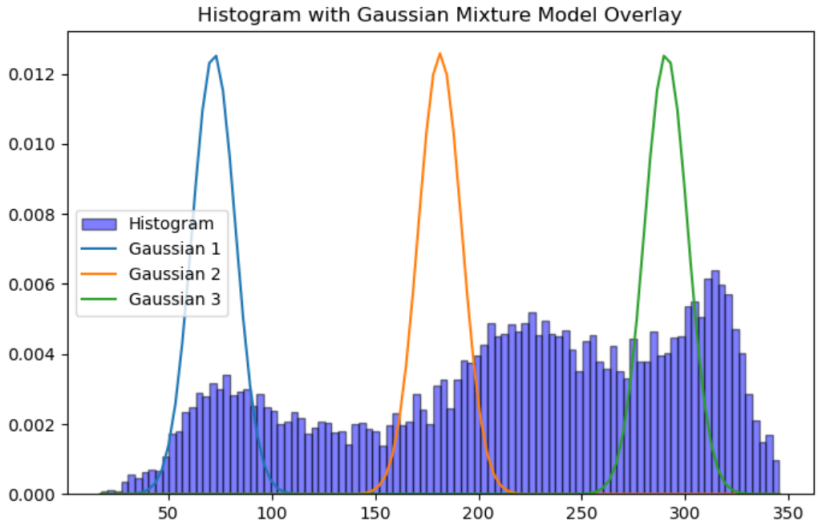


Figure 17: Histogram of the non-zero brain intensities with the three initialized Gaussian components overlaid. Each curve corresponds to one of the Gaussian distributions defined in Task 2, scaled by its mixing weight π_k

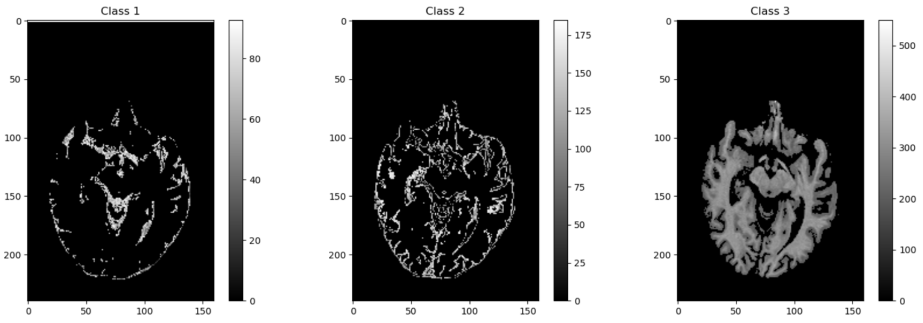


Figure 18: Hard segmentation obtained by assigning each pixel to the class with the highest weight $w_{n,k}$. Each subplot displays only the pixels belonging to one of the three classes, restricted to the brain mask.

Given the segmentation prior and likelihood model, the posterior distribution over labels can be obtained using Bayes' rule. Under the assumption of statistical independence between voxels, the posterior factorizes over voxels as

$$p(\mathbf{l} \mid \mathbf{d}, \hat{\boldsymbol{\theta}}) = \prod_{n=1}^{N} p(l_n \mid d_n, \hat{\boldsymbol{\theta}}), \tag{47}$$

where the per-voxel posterior is given by

$$p(l_n = k \mid d_n, \hat{\boldsymbol{\theta}}) = \frac{\mathcal{N}(d_n \mid \mu_k, \sigma_k^2)\, \pi_k}{\sum_{j=1}^{K} \mathcal{N}(d_n \mid \mu_j, \sigma_j^2)\, \pi_j}. \tag{48}$$

These posterior probabilities provide a soft segmentation of the image, where each voxel is assigned a probability of belonging to each tissue class. Since the posterior probabilities for a given voxel sum to one, they can be interpreted as class membership probabilities.

A hard segmentation can be obtained using the maximum a posteriori (MAP) estimate, which assigns each voxel to the label with the highest posterior probability:

$$\hat{l}_n = \arg\max_k \; p(l_n = k \mid d_n, \hat{\boldsymbol{\theta}}). \tag{49}$$

5.2 How to optimize parameters with the EM algorithm

The Expectation–Maximization (EM) algorithm is a key method for automatically delineating tissues from one another. This is achieved by estimating the parameters that maximize the likelihood function $p(\mathbf{d} \mid \boldsymbol{\theta})$, which expresses how probable the observed image $\mathbf{d}$ is for a given parameter vector $\boldsymbol{\theta}$:

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} p(\mathbf{d} \mid \boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \log p(\mathbf{d} \mid \boldsymbol{\theta}). \tag{50}$$

The logarithm can be used since it is a monotonically increasing function, meaning that the location of the maximum is unchanged. In addition, the log-likelihood simplifies the optimization since products become sums. The parameter vector $\hat{\boldsymbol{\theta}}$ is referred to as the maximum likelihood (ML) estimate.

Direct maximization of the log-likelihood is a non-trivial optimization problem due to the presence of latent variables, namely the unknown tissue labels. The EM algorithm addresses this by iteratively constructing and maximizing a lower bound on the log-likelihood until convergence.

The lower bound is defined through the Q-function. Given a current parameter estimate $\tilde{\boldsymbol{\theta}}$, the Q-function is defined as the expected complete-data log-likelihood:

$$\mathcal{Q}(\boldsymbol{\theta} \mid \tilde{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{l}|\mathbf{d},\tilde{\boldsymbol{\theta}}} \left[\log p(\mathbf{d}, \mathbf{l} \mid \boldsymbol{\theta}) \right]. \tag{51}$$

By construction, this function provides a lower bound on the log-likelihood,

$$\mathcal{Q}(\boldsymbol{\theta} \mid \tilde{\boldsymbol{\theta}}) \leq \log p(\mathbf{d} \mid \boldsymbol{\theta}), \tag{52}$$

with equality at $\boldsymbol{\theta} = \tilde{\boldsymbol{\theta}}$. Each EM iteration is guaranteed to increase the log-likelihood, and the algorithm converges to a local maximum.

Jensen's inequality is used to derive this lower bound. The bound becomes tight when the weights are chosen as the posterior probabilities of the labels given the current parameter estimate:

$$\omega_{n,k} = \frac{\mathcal{N}(d_n \mid \tilde{\mu}_k, \tilde{\sigma}_k^2)\, \tilde{\pi}_k}{\sum_{k'=1}^{K} \mathcal{N}(d_n \mid \tilde{\mu}_{k'}, \tilde{\sigma}_{k'}^2)\, \tilde{\pi}_{k'}}. \tag{53}$$

These weights correspond to the posterior probabilities of the tissue labels given the current model parameters. With this choice, the Q-function (lower bound) can be written as:

$$\begin{aligned} Q(\boldsymbol{\theta} \mid \hat{\boldsymbol{\theta}}) = -\frac{1}{2} \sum_{k=1}^{K} \left[\frac{1}{\sigma_k^2} \sum_{n=1}^{N} \omega_{n,k} \left(d_n - \mu_k\right)^2 + \left(\sum_{n=1}^{N} \omega_{n,k} \right) \log \sigma_k^2 \right] \\ + \sum_{k=1}^{K} \left[\left(\sum_{n=1}^{N} \omega_{n,k} \right) \log \pi_k \right] - \sum_{n=1}^{N} \sum_{k=1}^{K} \omega_{n,k} \log \omega_{n,k} - \frac{N}{2} \log(2\pi). \end{aligned} \tag{54}$$

The maximization step then updates the parameters by maximizing the Q-function, leading to the following update rules:

$$\tilde{\mu}_k \leftarrow \frac{\sum_{n=1}^{N} \omega_{n,k}\, d_n}{\sum_{n=1}^{N} \omega_{n,k}} \tag{55}$$

$$\tilde{\sigma}_k^2 \leftarrow \frac{\sum_{n=1}^{N} \omega_{n,k} \left(d_n - \tilde{\mu}_k\right)^2}{\sum_{n=1}^{N} \omega_{n,k}} \tag{56}$$

$$\tilde{\pi}_k \leftarrow \frac{\sum_{n=1}^{N} \omega_{n,k}}{N} \tag{57}$$

The EM algorithm then proceeds iteratively as follows:

1. Initialize the model parameters.
2. **E-step**: Estimate the weights $\omega_{n,k}$ using the current parameter values.
3. **M-step**: Update the model parameters using the current weights.
4. Repeat steps 2–3 until convergence.

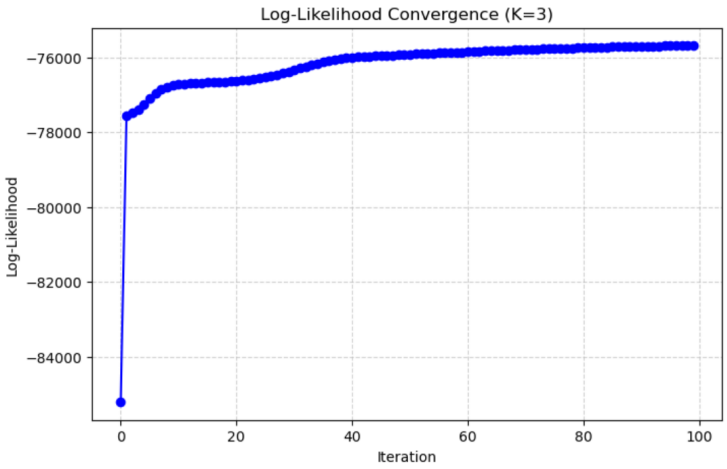


Figure 19: Evolution of the log-likelihood over EM iterations. The curve increases monotonically and plateaus after a number of iterations, indicating convergence of the parameter estimates.

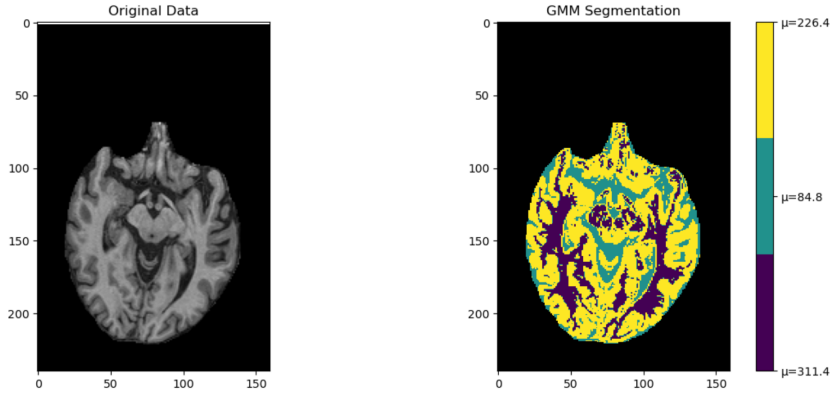


Figure 20: Final hard segmentation obtained from the converged responsibilities $w_{n,k}$. The left panel shows the original bias-corrected MR slice, and the right panel shows the final GMM-based segmentation. Each colour corresponds to one of the three tissue classes, with the colourbar indicating the class means mu_k in intensity units. Voxels outside the brain mask are shown in black.

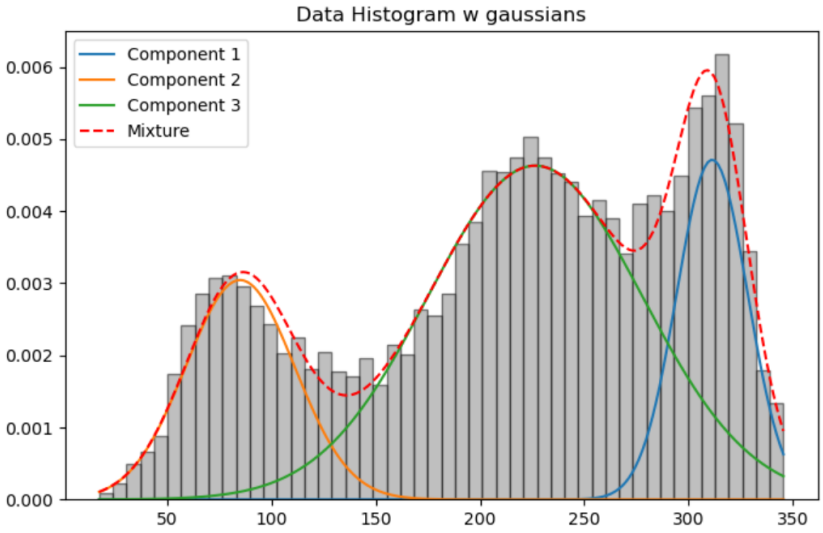


Figure 21: Corrected intensity histogram with the final Gaussian mixture model overlaid. The three weighted Gaussian components and their sum provide a good approximation of the empirical intensity distribution.

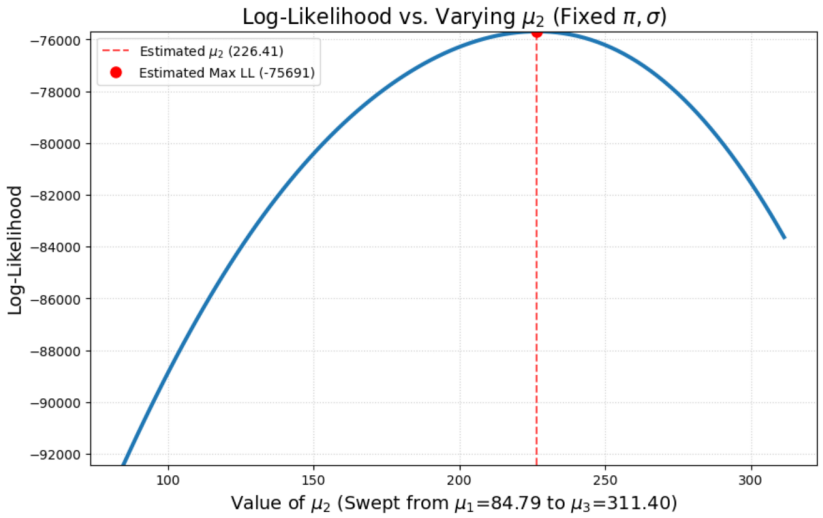


Figure 22: Log-likelihood as a function of μ_2 while keeping all other parameters fixed.

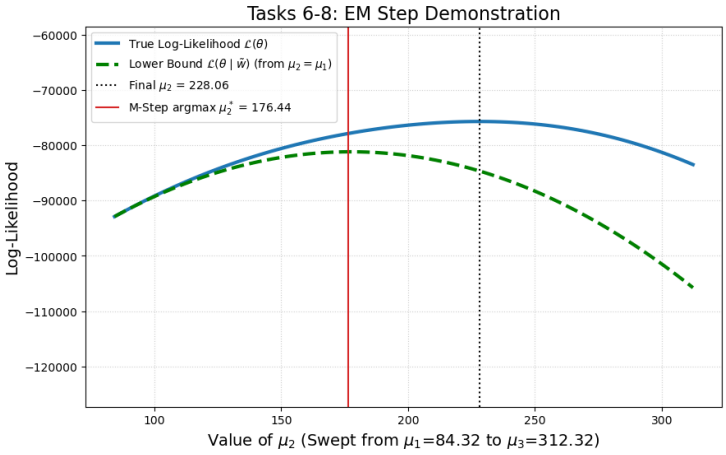


Figure 23: Lower Bound

6 Neural networks

The methods for registration and segmentation that have been described so far are based on models that encodes prior knownlegde of the at hand. For instance, mutual information-based registration exploits the fact that one image is predictive of another image only when the two are well aligned; while the Gaussian mixture model for segmentation encodes the knowledge that voxels with the same label typically have similar intensities.

Now we embark on a completely different approach to solve the problem of segmentation, we will use a supervised approach, data input/output $\{\mathbf{x}_n, t_n\}_{n=1}^N$ pairs, instead of unsupervised as in the GMM, where labels are unknown and we learn distributions that describes the data's labels. This models is called discriminative learning.

ADD A TRANSFER LINE TO NEXT SUBSECTION

6.1 Logistic Regression

In the Gaussian mixture model, Bayes' rule is used to obtain the probability that a voxel has label l, i.e. the posterior $p(l = k \mid d, \boldsymbol{\theta})$.

In contrast, instead of explicitly computing a voxel-wise probabilistic classifier, a neural network directly maps the input data to an output prediction. The predicted output is compared to the ground-truth using a similarity (loss) measure, and this information is used during backpropagation to compute the gradient of the loss with respect to the network parameters. The parameters are then updated by optimizing in the direction of steepest descent of the loss landscape.

To model $p(y \mid x, \boldsymbol{\theta})$—which is equivalent to modeling $p(l \mid d, \boldsymbol{\theta})$ for a voxel-wise classifier—is achieved using simple linear functions combined with a non-linear sigmoid function. This makes it possible to model complex nonlinear functions with arbitrary precision by combining small linear segments. This takes the form of a parametric curve:

$$f(\mathbf{x}) = \sigma\left(\sum_{m=0}^{M-1} w_m \phi_m(\mathbf{x})\right). \tag{58}$$

This is similar to the linear basis function model for regression used in smoothing and interpolation. It is a linear combination of M nonlinear basis functions $\phi_m(\mathbf{x})$ with tunable coefficients w_m, contained in the parameter vector $\boldsymbol{\theta} = (w_0, \cdots, w_{M-1})^T$. However, unlike the regression case, we do not use this linear combination directly; instead, we apply a sigmoid function.

$$a = \sum_{m=0}^{M-1} w_m \phi_m(\mathbf{x}) \tag{59}$$

and apply the sigmoid:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}. \tag{60}$$

The sigmoid maps the input into the interval $[0, 1]$. Because of this, $f(\mathbf{x})$ can be interpreted as a probability,

$$p(t = 1 \mid \mathbf{x}, \boldsymbol{\theta}) = f(\mathbf{x}),$$

since

$$p(t = 0 \mid \mathbf{x}, \boldsymbol{\theta}) = 1 - p(t = 1 \mid \mathbf{x}, \boldsymbol{\theta}) = 1 - f(\mathbf{x}).$$

This corresponds to a Bernoulli distribution:

$$p(t \mid \mathbf{x}, \boldsymbol{\theta}) = f(\mathbf{x})^t \left[1 - f(\mathbf{x})\right]^{1-t}. \tag{61}$$

The model is called logistic regression because of the sigmoid function σ. Given a training set of N input vectors $\mathbf{X} = \{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$ with corresponding outputs $\mathbf{t} = \{t_1, \cdots, t_N\}$, we estimate the parameter values $\hat{\boldsymbol{\theta}}$ by numerically maximizing the likelihood function:

$$p(\mathbf{t} \mid \mathbf{X}, \boldsymbol{\theta}) = \prod_{n=1}^{N} p(t_n \mid \mathbf{x}_n, \boldsymbol{\theta}). \tag{62}$$

with respect to θ. Once the model is trains/fitted, we can subsequently use the model to act as a classifier for new voxel by simply:

$$p(l = 1 | d, \hat{\theta}) = p(t = 1 | \mathbf{x}, \hat{\theta}) = \sigma\left(\sum_{m=0}^{M-1} \hat{w}_m \phi_m(\mathbf{x})\right) \tag{63}$$

We assign a voxel to class 1 if $p(l = |\mathbf{x}, \hat{\theta}) > 0.5$, else to class 2.

6.2 How to train a neural network?

To train a neural network (NN), we need an optimization criterion. This is usually done by maximizing the likelihood. However, for neural networks, we typically formulate the problem as a minimization problem, so that we can find the parameters where the gradient equals zero. Therefore, we use the negative log-likelihood, since minimizing this is equivalent to maximizing the log-likelihood.

A commonly used optimization criterion is known as cross-entropy, which we want to minimize:

$$E_N(\boldsymbol{\theta}) = -\log p(\mathbf{t} \mid \mathbf{X}, \boldsymbol{\theta}) = -\sum_{n=1}^{N} \{t_n \log f(x_n) + (1 - t_n) \log [1 - f(x_n)]\} \tag{64}$$

Starting from randomly initialized parameters $\boldsymbol{\theta}$, standard gradient descent proceeds by iteratively stepping in the direction of steepest descent, which is calculated using the whole input batch:

$$\boldsymbol{\theta}^{(\tau+1)} = \boldsymbol{\theta}^{(\tau)} - v \nabla E_N(\boldsymbol{\theta}^{(\tau)}) \tag{65}$$

Here, τ denotes the iteration number, $\nabla E_N(\boldsymbol{\theta}) = \frac{dE_N}{d\boldsymbol{\theta}}$ is the gradient of the energy function with respect to the parameters, and v is the step size (learning rate). However, this method is inefficient for large datasets. Therefore, stochastic gradient descent (SGD) is introduced, where the gradient is computed using a random subset of the input batch.

This has two important features. First, it introduces stochasticity into the loss landscape, causing small variations in each iteration, which helps the optimizer explore different regions of the loss surface. Second, it is computationally more efficient.

$$\nabla E_N(\boldsymbol{\theta}) = \frac{N}{N'} \nabla E_{N'}(\boldsymbol{\theta}) \tag{66}$$

where the update rule is given by:

$$\boldsymbol{\theta}^{(\tau+1)} = \boldsymbol{\theta}^{(\tau)} - v' \nabla E_{N'}(\boldsymbol{\theta}^{(\tau)}) \tag{67}$$

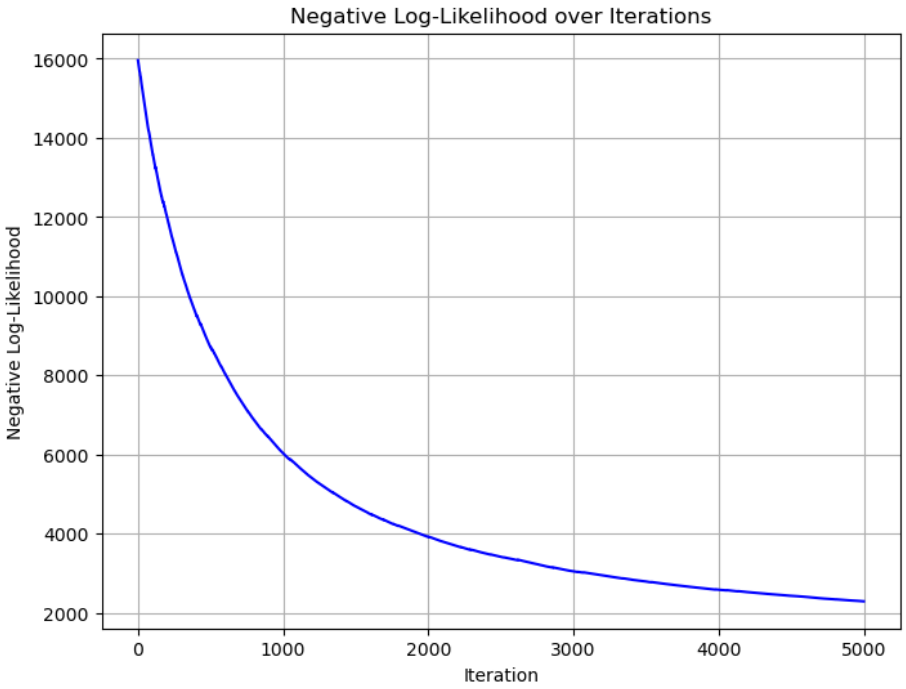


Figure 24: Cross-entropy over iterations

Figure 25 shows the final segmentation.

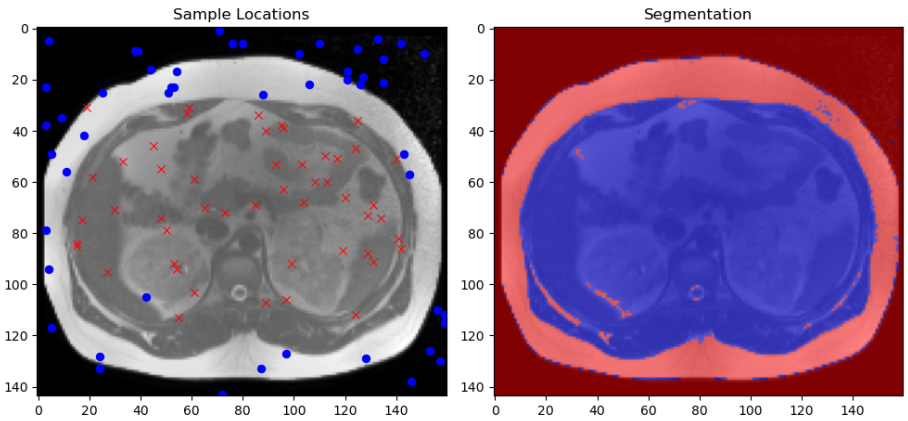


Figure 25: Final segmentation and sample locations

6.3 9D input

We can add learnable parameters to our basis functions so that they themselves become adaptive. This allows the model to learn what it should focus on. These parameters are updated using SGD during backpropagation. The adaptive basis functions are given by:

$$\phi_m(\mathbf{x}) = \begin{cases} 1, & \text{if } m = 0, \\ \sigma\left(\sum_{d=1}^{D} \beta_{m,d} x_d + \beta_{m,0}\right), & \text{otherwise.} \end{cases} \tag{68}$$

Here, σ is the logistic sigmoid function, and the β parameters are additional learnable parameters that, together with w, are contained in $\boldsymbol{\theta}$.

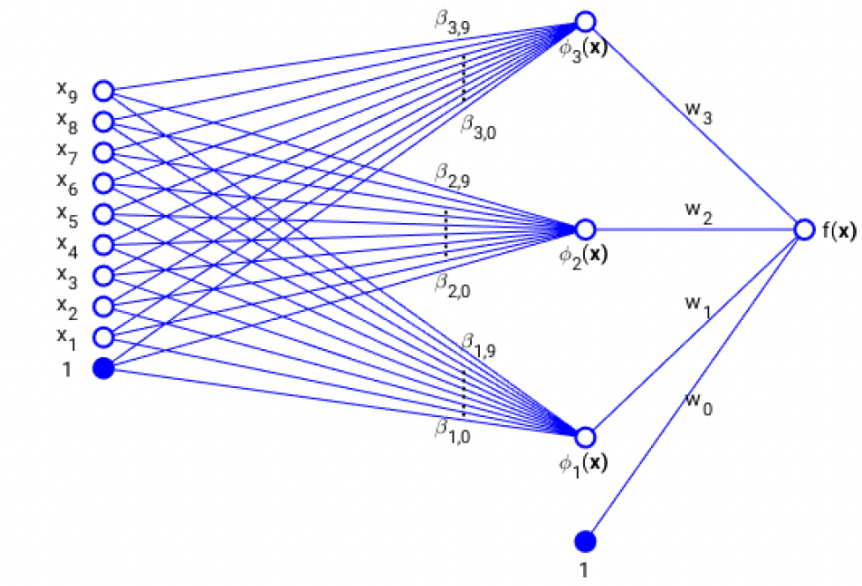


Figure 26: Feed-forward neural network

In the image above, we see how the input is propagated through the network. If the number of basis functions is $M = 4$ and the input dimension is $D = 9$, then all 9 input data points are passed to the 3 nonlinear basis functions. The first basis function corresponds to the intercept and does not receive input data, but it is still a learnable parameter.

The parameters $\boldsymbol{\beta}_m$ are illustrated in Figure 30. They represent the weights applied to each input dimension across the image for the different basis functions. When applied to the whole image, they produce feature maps (see Figure 29), which contain information that the network finds important for the given task. For example, one feature map may have high intensity in the background

and low intensity in the foreground, while another may focus on edges or boundaries. A third feature map may contain little useful information for the given task.

Adaptive basis functions

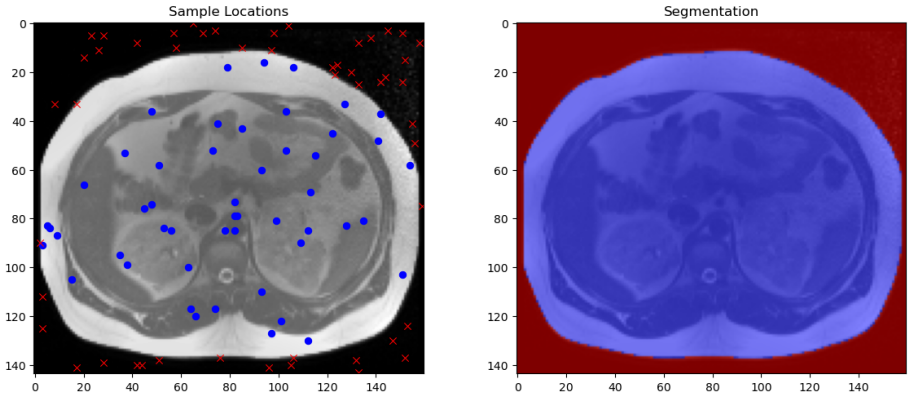


Figure 27: Final segmentation when using adaptive basis functions.

Figure 28 visualizes the learned adaptive basis functions.

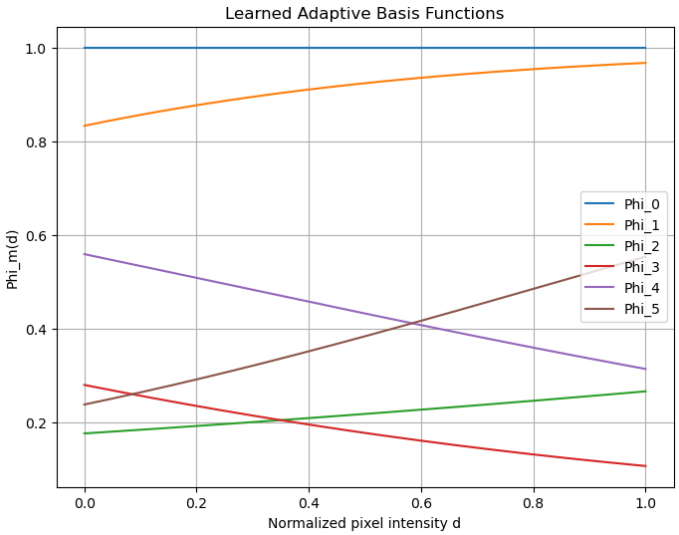


Figure 28: Learned adaptive basis functions

3x3 filter 8 neighbors

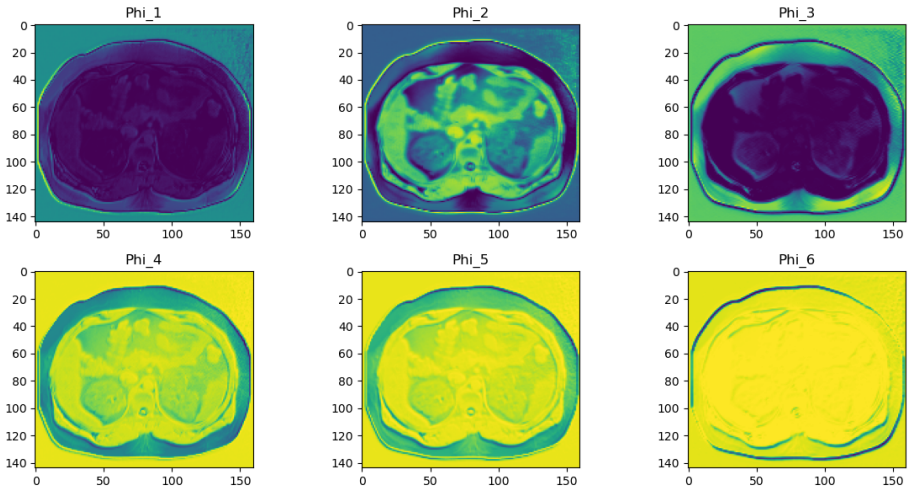


Figure 29: Feature maps used to generate the segmentation

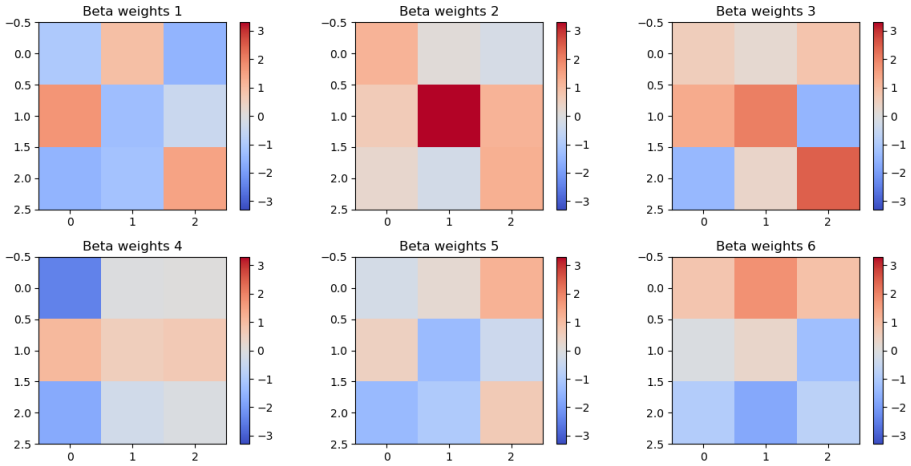


Figure 30: Weight maps used to generate the feature maps.